

Towards the 3D world

①

- ① Representing the user's view into the 3D world.
- ② Controlling the volume of 3D space that is viewed.
- ③ Clipping
- ④ Handling Foreground & Background object.
- ⑤ Drawing a 3D object (cube).

① The only difference from Part ② is that, we need to consider the depth information, in addition to 'x' & 'y' coordinates.

Specifying the Viewing direction
3D objects has depth so one difference is that you can view object from anywhere in space.

Two important points

① Viewing direction



- (i) Where you are looking from
- (ii) Which part of the scene you are looking

② The Visible Range.



Given the viewing direction where you can actually see.

Placing the eye point anywhere in ③
3D Space of looking objects from
Various direction.

Remember So far eye point was placed
at origin $(0, 0, 0)$ of the line
of sight in the negative direction.
↓
(inward towards your computer screen)

Lets move the eye point / camera from
default location to other location.

EYE POINT, LOOK-AT-POINT, & UP
DIRECTION.

Three key information are required
for "Which Part of scene" & where you
are looking from.

EYE POINT - where you are looking from
LOOK UP " - which part of scene you are look
DIRECTION UP IN SCENE - Direction of the scene.

EYE POINT

④

① Starting Point from which 3D space is viewed.

Coordinates of this position are (eye X, eye Y, eye Z).

Look-at-Point

Direction of the line of sight from the eye point.

Eye point is not a vector, it's a point, so another point is required ~~for~~ for direction you are looking.

"The lookUp point is a point on the line of sight extending from the eye point.

its coordinates are

(at X, at Y, at Z).

UP direction

(5)

This determines the UP direction in the scene, that is being viewed from the eye point to the look-at-point.

"To define the rotation you must determine the UP-direction along the line of sight."

The coordinates of this direction are (UP_X, UP_Y, UP_Z) .

How to specify the position & direction.

[Using a matrix & passing the matrix to vertex shader.]

↓
Why Matrix?

"View Transformation" or "view matrix"

because it changes the view of the scene.

→ (Moving inward is similar to Translation in z-axis)
"Remember Model Matrix"?

cuon-matrix.js, the method

⑥

Matrix4.setLookAt() is defined to calculate the View matrix from the three items of information.

Matrix4.setLookAt (eyeX, eyeY, eyeZ, atX, atY, atZ
UPX, UPY, UPZ

var initialViewMatrix = new Matrix4();

initialViewMatrix.setLookAt(0,0,0,0,1,0,0,1,0);

LookAtTriangles.js

Two differences -

- ① Z-coordinate added in vertices
- ② ViewMatrix.setLookAt method.

Similarities B/w Rotation & Viewing.

⑦

The `SetLookAt()` method of the `Matrix4` Object just calculate the matrix to carry out this transformation using the information about the position of the eye-point, look & up direction.

LookAt Rotated Triangles.js.

We can rotate like this

```
Var modelViewMatrix = new Matrix4()
```

```
ModelViewMatrix.setLookAt(0.20, 0.25, 0.25, 0, 0, 0,  
                           0, 1, 0)
```

.rotate

```
(-10, 0, 0, 1);
```

Pass the matrix to `uniform` variable.

```
gl.uniformMatrix4fv(u_Model, false, modelViewMatrix)
```